МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №4
Варіант 4

на тему «Рекурсія та мемоізація в Python»

Виконав:
студент групи ФеП-22
Корецький Андрій Вікторович

Перевірив:
аспірант
Остріков Данило Андрійович

Львів – 2026

Мета роботи. зрозуміти рекурсивні обчислення, обмеження стеку Python та відсутність TCO; навчитися мемоізувати обчислення за допомогою functools.lru_cache; реалізувати рекурсивний бінарний пошук, quicksort, обходи дерев і memoized Fibonacci.

Теоретичні відомості

Рекурсія — функція викликає саму себе для меншої підзадачі. Обов'язкові елементи: **база рекурсії** (простий випадок) і **рекурсивний крок** (зменшення задачі).

CPython обмежує глибину стеку викликів (sys.getrecursionlimit() ≈ 1000); перевищення призводить до RecursionError. **Оптимізації хвостових викликів (TCO) у CPython немає**, тому навіть хвостова рекурсія росте у стеку; для глибоких задач безпечніші ітеративні алгоритми.

Мемоізація (@functools.lru_cache) кешує результат для однакових аргументів і перетворює експоненційні рекурсії (Фібоначчі) на лінійні. cache_info() показує статистику, cache_clear() очищує кеш.

Індивідуальне завдання (варіант 4)

Тема «Рекурсивний пошук у списку». Реалізувати рекурсивний бінарний пошук у відсортованій незмінній послідовності; порівняти з ітеративним пошуком; протестувати знайдені та відсутні значення.

Хід роботи

1. Рекурсивний бінарний пошук

База рекурсії: порожній діапазон (lo > hi) → −1 або знайдений елемент → mid. Крок: пошук у лівій або правій половині. Послідовність не копіюється (передаються лише межі), тому глибина рекурсії O(log n): для мільйона елементів — 20 кадрів.

```
def binary_search_recursive(
    seq: Sequence[T], target: T, lo: int = 0, hi: int | None = None
) -> int:
    """Індекс ``target`` у відсортованій ``seq`` або -1.

    База рекурсії:  порожній діапазон (lo > hi) -> -1; seq[mid] == target -> mid.
    Крок рекурсії:  шукаємо в лівій АБО правій половині (задача зменшується вдвічі).
    Послідовність не копіюється й не змінюється — передаються лише межі lo/hi,
    тому глибина рекурсії O(log n), а пам'ять O(log n) на стек.
    """
    if hi is None:
        hi = len(seq) - 1
```

```
    if lo > hi:  # база: діапазон порожній
        return -1
    mid = (lo + hi) // 2
    value = seq[mid]
    if target < value:
        return binary_search_recursive(seq, target, lo, mid - 1)
    if value < target:
        return binary_search_recursive(seq, target, mid + 1, hi)
    return mid  # база: знайдено
```

Лістинг 1 — фрагмент файлу algorithms.py

2. Ітеративна альтернатива та мемоізація

Ітеративна версія використовує O(1) пам'яті та не залежить від ліміту рекурсії. binary_search_cached показує застосування lru_cache: tuple хешується, тому (seq, target) є ключем кешу.

```
def binary_search_iterative(seq: Sequence[T], target: T) -> int:
    """Ітеративна альтернатива: той самий алгоритм, але O(1) додаткової пам'яті."""
    lo, hi = 0, len(seq) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        value = seq[mid]
        if target < value:
            hi = mid - 1
        elif value < target:
            lo = mid + 1
        else:
            return mid
    return -1

@lru_cache(maxsize=1024)
def binary_search_cached(seq: tuple[int, ...], target: int) -> int:
    """Мемоізований пошук: tuple хешується, тож (seq, target) — ключ кешу.
    Виграш — для повторних запитів того самого значення в тій самій послідовності.
    Демонструє cache_info() / cache_clear().
    """
    return binary_search_recursive(seq, target)
```

Лістинг 2 — фрагмент файлу algorithms.py

3. Перше входження, trampoline і ліміт стеку

lower_bound_recursive знаходить перше входження серед дублікатів. Trampoline моделює TCO: функція повертає «наступний крок» як lambda, а цикл виконує кроки без росту стеку. linear_search_recursive має глибину O(n) і на 5000 елементах дає RecursionError.

```
def lower_bound_recursive(
    seq: Sequence[T], target: T, lo: int = 0, hi: int | None = None
) -> int:
    """Перша позиція, куди можна вставити ``target`` (аналог bisect_left).

    Корисно для послідовностей з дублікатами: повертає ПЕРШЕ входження.
    """
    if hi is None:
        hi = len(seq)
    if lo >= hi:
        return lo
    mid = (lo + hi) // 2
```

```
    if seq[mid] < target:
        return lower_bound_recursive(seq, target, mid + 1, hi)
    return lower_bound_recursive(seq, target, lo, mid)

def trampoline(result: Any) -> Any:
    """Виконує «кроки», поки функція повертає thunk (callable без аргументів)."""
    while callable(result):
        result = result()
    return result

def binary_search_tramp(seq: Sequence[T], target: T, lo: int, hi: int) -> Any:
    """Бінарний пошук у стилі trampoline: замість виклику повертає lambda."""
    if lo > hi:
        return -1
    mid = (lo + hi) // 2
    if target < seq[mid]:
        return lambda: binary_search_tramp(seq, target, lo, mid - 1)
    if seq[mid] < target:
        return lambda: binary_search_tramp(seq, target, mid + 1, hi)
    return mid
```

Лістинг 3 — фрагмент файлу algorithms.py

4. Базова частина: quicksort, дерева, Фібоначчі

Реалізовано quicksort з випадковим pivot, 3-шляховий і гібридний варіанти, рекурсивні обходи дерева, ітеративні BFS/DFS, висоту дерева, fib з lru_cache і fib_iter.

```
def quicksort(xs: Sequence[T], rng: random.Random | None = None) -> list[T]:
    """Чистий рекурсивний quicksort з випадковим pivot (вхід не змінюється)."""
    if len(xs) <= 1:  # база
        return list(xs)
    pivot = (rng or random).choice(xs)
    left = [x for x in xs if x < pivot]
    mid = [x for x in xs if not (x < pivot) and not (pivot < x)]
    right = [x for x in xs if pivot < x]
    return quicksort(left, rng) + mid + quicksort(right, rng)

@cache
def fib(n: int) -> int:
    """Мемоізований Фібоначчі: O(n) часу, O(n) глибини стеку."""
    if n < 0:
        raise ValueError("n must be non-negative")
    return n if n < 2 else fib(n - 1) + fib(n - 2)

def fib_iter(n: int) -> int:
    """DP без рекурсії — безпечно для будь-якого n."""
    if n < 0:
        raise ValueError("n must be non-negative")
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

Лістинг 4 — фрагмент файлу algorithms.py

5. Тестування

42 тести: кожен елемент знаходиться, відсутні значення, порожня та одноелементна послідовності, рядки, дублікати, порівняння з ітеративною версією на випадкових даних, логарифмічна глибина, RecursionError для лінійної рекурсії, trampoline, cache_info / cache_clear, а також тести quicksort, дерев і Фібоначчі.

```
@pytest.mark.parametrize("missing", [-5, 0, 3, 13, 57, 92, 1000])
def test_missing_values(missing: int) -> None:
    assert binary_search_recursive(SEQ, missing) == -1
    assert binary_search_iterative(SEQ, missing) == -1

def test_cache_info_and_clear() -> None:
    binary_search_cached.cache_clear()
    binary_search_cached(SEQ, 23)
    binary_search_cached(SEQ, 23)
    binary_search_cached(SEQ, 24)
    info = binary_search_cached.cache_info()
    assert (info.hits, info.misses, info.currsize) == (1, 2, 2)
    binary_search_cached.cache_clear()
    assert binary_search_cached.cache_info().currsize == 0
```

Лістинг 5 — фрагмент файлу tests/test_search.py

Результати виконання

Результат запуску програми командою `python main.py`:

```
> python main.py
=== 1. Бінарний пошук у незмінній відсортованій послідовності ===
Послідовність: (15, 23, 42, 42, 42, 57, 64, 78, 91, 105)
  шукаємо  57: recursive= 5  iterative= 5  глибина=3  (знайдено)
  шукаємо  15: recursive= 0  iterative= 0  глибина=3  (знайдено)
  шукаємо 105: recursive= 9  iterative= 9  глибина=4  (знайдено)
  шукаємо  42: recursive= 4  iterative= 4  глибина=1  (знайдено)
  шукаємо  50: recursive=-1  iterative=-1  глибина=4  (відсутнє)
  шукаємо   0: recursive=-1  iterative=-1  глибина=4  (відсутнє)
  шукаємо 200: recursive=-1  iterative=-1  глибина=5  (відсутнє)
  перше входження 42 (lower_bound): 2
  рядки: ('apple', 'banana', 'cherry', 'kiwi', 'mango') -> 'kiwi' на індексі 3
  1 000 000 елементів: індекс=999999, глибина рекурсії=20 (≈ log2 n)
  trampoline-версія: 999999
  lru_cache: CacheInfo(hits=2, misses=3, maxsize=1024, currsize=3)

=== 2. Ліміт рекурсії та відсутність TCO ===
  sys.getrecursionlimit() = 1000
  linear_search_recursive на 5000 елементів -> RecursionError
  binary_search_recursive на тих самих даних -> 4999
  fact_tail(5000) (хвостова рекурсія) -> RecursionError: TCO немає
  trampoline(fact_tramp(5000)) має 54233 біт

=== 3. Quicksort ===
  вхід: [5, 3, 8, 1, 9, 2, 5, 5, 0]
  quicksort: [0, 1, 2, 3, 5, 5, 5, 8, 9]
  quicksort_3way: [0, 1, 2, 3, 5, 5, 5, 8, 9]
  вхід після сортування (не змінився): [5, 3, 8, 1, 9, 2, 5, 5, 0]

=== 4. Дерево ===
  preorder : [4, 2, 1, 3, 6, 7]
  inorder  : [1, 2, 3, 4, 6, 7]
  postorder: [1, 3, 2, 7, 6, 4]
  BFS      : [4, 2, 6, 1, 3, 7]
  DFS iter : [4, 2, 1, 3, 6, 7]
  висота rec/iter: 3 3
```

```
  BST з tuple, inorder: [10, 20, 30, 40, 50, 60, 70] | містить 50: True

=== 5. Фібоначчі ===
  fib(0..10): [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
  fib(90) == fib_iter(90): True
  fib.cache_info(): CacheInfo(hits=99, misses=91, maxsize=None, currsize=91)
```

Рисунок 1 — результат роботи програми (Lab_04)

Результат автоматизованого тестування `pytest -v` та статичних перевірок:

```
> black --check .
All done!
> ruff check .
All checks passed!
> mypy .
Success: no issues found
> pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.3, pytest-9.2.0, pluggy-1.7.0 -- C:\labs\Функціональне
    програмування\.venv\Scripts\python.exe
rootdir: C:\labs\Функціональне програмування\functional-programming\Lab_04
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.14.5
collecting ... collected 42 items
tests/test_fib.py::test_fib_small PASSED                                 [  2%]
tests/test_fib.py::test_fib_iter_matches PASSED                          [  4%]
tests/test_fib.py::test_cache_clear_and_info PASSED                      [  7%]
tests/test_fib.py::test_negative_input PASSED                            [  9%]
tests/test_fib.py::test_tail_recursion_no_tco_and_trampoline PASSED      [ 11%]
tests/test_performance.py::test_search_recursive_vs_iterative PASSED     [ 14%]
tests/test_performance.py::test_fib_memoization_speedup PASSED           [ 16%]
tests/test_quicksort.py::test_quicksort_basic PASSED                     [ 19%]
tests/test_quicksort.py::test_quicksort_random_against_sorted PASSED     [ 21%]
tests/test_quicksort.py::test_many_duplicates_3way PASSED                [ 23%]
tests/test_quicksort.py::test_quicksort_does_not_mutate PASSED           [ 26%]
tests/test_quicksort.py::test_insertion_and_median PASSED                [ 28%]
tests/test_search.py::test_found_every_element[0-2] PASSED               [ 30%]
tests/test_search.py::test_found_every_element[1-5] PASSED               [ 33%]
tests/test_search.py::test_found_every_element[2-8] PASSED               [ 35%]
tests/test_search.py::test_found_every_element[3-12] PASSED              [ 38%]
tests/test_search.py::test_found_every_element[4-16] PASSED              [ 40%]
tests/test_search.py::test_found_every_element[5-23] PASSED              [ 42%]
tests/test_search.py::test_found_every_element[6-38] PASSED              [ 45%]
tests/test_search.py::test_found_every_element[7-56] PASSED              [ 47%]
tests/test_search.py::test_found_every_element[8-72] PASSED              [ 50%]
tests/test_search.py::test_found_every_element[9-91] PASSED              [ 52%]
tests/test_search.py::test_missing_values[-5] PASSED                     [ 54%]
tests/test_search.py::test_missing_values[0] PASSED                      [ 57%]
tests/test_search.py::test_missing_values[3] PASSED                      [ 59%]
tests/test_search.py::test_missing_values[13] PASSED                     [ 61%]
tests/test_search.py::test_missing_values[57] PASSED                     [ 64%]
tests/test_search.py::test_missing_values[92] PASSED                     [ 66%]
tests/test_search.py::test_missing_values[1000] PASSED                   [ 69%]
tests/test_search.py::test_edge_cases_empty_and_single PASSED            [ 71%]
tests/test_search.py::test_first_and_last_and_strings PASSED             [ 73%]
tests/test_search.py::test_input_is_not_mutated PASSED                   [ 76%]
tests/test_search.py::test_duplicates_lower_bound PASSED                 [ 78%]
tests/test_search.py::test_recursive_matches_iterative_random PASSED     [ 80%]
tests/test_search.py::test_depth_is_logarithmic PASSED                   [ 83%]
tests/test_search.py::test_linear_recursion_hits_limit_but_binary_does_not PASSED [ 85%]
tests/test_search.py::test_trampoline_search PASSED                      [ 88%]
tests/test_search.py::test_cache_info_and_clear PASSED                   [ 90%]
tests/test_tree.py::test_traversals PASSED                               [ 92%]
tests/test_tree.py::test_empty_tree PASSED                               [ 95%]
tests/test_tree.py::test_height PASSED                                   [ 97%]
tests/test_tree.py::test_build_bst_and_search PASSED                     [100%]
============================== 42 passed in 0.21s ==============================
```

Рисунок 2 — результати тестів і перевірок якості коду (Lab_04)

Порівняння часу виконання (python benchmark.py):

```
     size |  recursive |  iterative | cached warm | bisect
      100 |    0.00135 |    0.00107 |     0.00015 | 0.00024
    10000 |    0.00442 |    0.00255 |     0.00342 | 0.00046
  1000000 |    0.00867 |    0.00614 |     0.00718 | 0.00190
(час у секундах на 2000 пошуків)
fib(25): naive=0.01298s  lru_cache=0.000042s  iter=0.000004s
```

Рисунок 3 — порівняння часу виконання (python benchmark.py)

Висновок

Реалізовано чистий рекурсивний бінарний пошук у незмінній відсортованій послідовності з чіткими базовими випадками та глибиною рекурсії O(log n), а також ітеративну альтернативу. Вимірювання показали, що ітеративна версія швидша приблизно на 30–40 % через відсутність накладних витрат на кадри стеку, хоча асимптотика однакова. Продемонстровано ліміт рекурсії CPython і відсутність TCO (хвостова рекурсія fact_tail падає з RecursionError), а також обхід цього обмеження через trampoline. Мемоізація через lru_cache прискорила обчислення Фібоначчі на кілька порядків.